

1. soundfile

soundfile 라이브러리로 wav 파일의 fmt chunk 관련 정보(sampling rate, channels, subtype 등)를 확인

1) Python 기본 wave 라이브러리로 BlockAlign, ByteRate 확인 시도 → 일부 파일에서 “unknown format: 3” 오류 발생

```
(base) PS C:\Users\sp9gh\Downloads> python .\check_audio_format.py
Traceback (most recent call last):
  File "C:\Users\sp9gh\Downloads\check_audio_format.py", line 19, in <module>
    with wave.open(path, "rb") as w:
         ~~~~~^~~~~~
  File "C:\Users\sp9gh\anaconda3\Lib\wave.py", line 661, in open
    return Wave_read(f)
  File "C:\Users\sp9gh\anaconda3\Lib\wave.py", line 286, in __init__
    self.initfp(f)
         ~~~~~^
  File "C:\Users\sp9gh\anaconda3\Lib\wave.py", line 266, in initfp
    self._read_fmt_chunk(chunk)
         ~~~~~^
  File "C:\Users\sp9gh\anaconda3\Lib\wave.py", line 387, in _read_fmt_chunk
    raise Error('unknown format: %r' % (wFormatTag,))
wave.Error: unknown format: 3
(base) PS C:\Users\sp9gh\Downloads>
```

2) format 3은 IEEE float WAV 형식이라 기본 wave로 처리하기 어렵기 때문에 soundfile로 subtype 분포를 확인하는 방식으로 변경

```
=== Sampling rate 분포 ===
sampling_rate
44100      5370
48000      2502
96000       610
24000       82
16000       45
22050       44
11025       39
192000      17
8000        12
11024       7
32000       4
Name: count, dtype: int64

=== Channel 분포 ===
channels
2      7993
1      739
Name: count, dtype: int64

=== Subtype 분포 ===
subtype
PCM_16      5758
PCM_24      2753
FLOAT       169
PCM_U8       43
MS_ADPCM      8
IMA_ADPCM      1
Name: count, dtype: int64
```

```

=== Format 분포 ===
format
WAV          5979
WAVEX        2753
Name: count, dtype: int64

=== Duration 요약 ===
count      8732.000000
mean        3.607522
std         0.974394
min         0.050000
25%         4.000000
50%         4.000000
75%         4.000000
max         4.036647
Name: duration_sec, dtype: float64

검사 완료! urbansound8k_audio_format_check.csv 파일이 생성되었습니다.
(base) PS C:\Users\sp9gh\Downloads> |

```

• 결과

1. Sampling rate 분포

가장 많이 사용된 sampling rate는:

Sampling rate	파일 수
44100 Hz	5370
48000 Hz	2502
96000 Hz	610

그 외에도:

24000

16000

22050

11025

192000

8000

등 다양한 sampling rate가 존재함.

- 데이터셋 내 sampling rate가 통일되어 있지 않음
- 모델 입력 shape 및 시간축 해석 통일을 위해 resampling 필요
- 일반적으로 16000Hz 또는 22050Hz 등으로 통일 가능

2. Channel 분포

Channels	파일 수
----------	------

Stereo(2채널)	7993
Mono(1채널)	739

- 대부분 stereo 파일
 - 일부 mono 파일 존재
 - 입력 형식 통일을 위해 mono 변환 고려 가능
- (Python)
- ```
waveform = waveform.mean(dim=0, keepdim=True)
```

### 3. Subtype(비트 저장 형식) 분포

| Subtype   | 파일 수 |
|-----------|------|
| PCM_16    | 5758 |
| PCM_24    | 2753 |
| FLOAT     | 169  |
| PCM_U8    | 43   |
| MS_ADPCM  | 8    |
| IMA_ADPCM | 1    |

- bit depth 및 encoding 방식이 서로 다름
- PCM\_16과 PCM\_24가 가장 많음
- FLOAT 및 ADPCM 형식도 일부 존재
- Python 기본 wave 라이브러리에서 일부 파일이 unknown format: 3 오류 발생
- 전처리 시 저장 형식 통일 필요 가능성 있음

### 4. Format 분포

| Format | 파일 수 |
|--------|------|
| WAV    | 5979 |
| WAVEX  | 2753 |

- 일반 WAV 외에도 WAVEX 형식 존재
- WAVEX는 확장 WAV 형식으로 추가 메타데이터를 포함 가능

### 5. Duration(길이) 분포

| 항목 | 값 |
|----|---|
|----|---|

|     |            |
|-----|------------|
| 평균  | 약<br>3.61초 |
| 중앙값 | 4초         |
| 최대  | 약<br>4.04초 |

- 대부분 파일 길이는 약 4초로 비교적 일정
- 일부 짧은 파일 존재
- 필요 시 padding(무음 추가)/truncation(잘라내기) 가능

#### • 전처리 방향 제안

##### ① Sampling rate 통일

예: 44100 / 48000 / 96000 ...

→ 16000Hz 또는 22050Hz로 resampling

##### ② Channel 통일

stereo → mono 변환

##### ③ Feature extraction

후보:

- MFCC
- Mel-spectrogram
- Chroma feature
- Spectral contrast

##### ④ 입력 길이 통일

- padding
- truncation

## 2. torchaudio

1) torchaudio를 이용한 preprocessing 테스트 수행을 시도했으나, 최신 torchaudio.load()는 torchcodec / FFmpeg dependency 문제로 Windows 환경에서 오류 발생

```
(base) PS C:\Users\sp9gh\Downloads> python .\test_torchaudio.py
테스트 파일: 101415-3-0-2.wav
Traceback (most recent call last):
 File "C:\Users\sp9gh\Downloads\test_torchaudio.py", line 14, in <module>
 waveform, sample_rate = torchaudio.load(path)
                             ~~~~~^~~~~~
  File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchaudio\__init__.py", line 86, in load
    return load_with_torchcodec(
           ~~~~~^~~~~~
 uri,
 ...<6 lines>...
 backend=backend,
)
 File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchaudio\torchcodec.py", line 82, in load_with_torchcodec
 from torchcodec.decoders import AudioDecoder
 File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec__init__.py", line 12, in <module>
 from . import decoders, encoders, samplers, transforms # noqa
    ~~~~~^~~~~~
  File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec\decoders\__init__.py", line 7, in <module>
    from .._core import AudioStreamMetadata, VideoStreamMetadata
  File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec\_core\__init__.py", line 8, in <module>
    from ._metadata import (
    ...<5 lines>...
    )
  File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec\_core\_metadata.py", line 15, in <module>
    from torchcodec._core.ops import (
    ...<3 lines>...
    )
  File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec\_core\ops.py", line 38, in <module>
    load_torchcodec_shared_libraries()
    ~~~~~^~~~~~
```

```

RuntimeError: Could not load libtorchcodec. Likely causes:
 1. FFmpeg is not properly installed in your environment. We support
 versions 4, 5, 6, 7, and 8, and we attempt to load libtorchcodec
 for each of those versions. Errors for versions not installed on
 your system are expected; only the error for your installed FFmpeg
 version is relevant. On Windows, ensure you've installed the
 "full-shared" version which ships DLLs.
 2. The PyTorch version (2.12.0+cpu) is not compatible with
 this version of TorchCodec. Refer to the version compatibility
 table:
 https://github.com/pytorch/torchcodec?tab=readme-ov-file#installing-torchcodec.
 3. Another runtime dependency; see exceptions below.

The following exceptions were raised as we tried to load libtorchcodec:

[start of libtorchcodec loading traceback]
FFmpeg version 8:
Traceback (most recent call last):
 File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torch_ops.py", line 1509, in load_library
 ctypes.CDLL(path)
    ~~~~~^~~~~~
  File "C:\Users\sp9gh\anaconda3\Lib\ctypes\__init__.py", line 390, in __init__
    self._handle = _dlopen(self._name, mode)
    ~~~~~^~~~~~
FileNotFoundError: Could not find module 'C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec\libtorchcodec_core8.dll' (or one of its dependencies). Try using the full path with constructor syntax.

```

```

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
 File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec_internally_replaced_utils.py", line 93, in load_torchcodec_shared_libraries
 torch.ops.load_library(core_library_path)
    ~~~~~^~~~~~
  File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torch\_ops.py", line 1511, in load_library
    raise OSError(f"Could not load this library: {path}") from e
OSError: Could not load this library: C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec\libtorchcodec_core8.dll

FFmpeg version 7:
Traceback (most recent call last):
  File "C:\Users\sp9gh\anaconda3\Lib\site-packages\torch\_ops.py", line 1509, in load_library
    ctypes.CDLL(path)
    ~~~~~^~~~~~
 File "C:\Users\sp9gh\anaconda3\Lib\ctypes__init__.py", line 390, in __init__
 self._handle = _dlopen(self._name, mode)
    ~~~~~^~~~~~
FileNotFoundError: Could not find module 'C:\Users\sp9gh\anaconda3\Lib\site-packages\torchcodec\libtorchcodec_core7.dll' (or one of its dependencies). Try using the full path with constructor syntax.

```

2) soundfile로 WAV 파일을 읽은 뒤 torch tensor로 변환하고 torchaudio.transforms.Resample를 적용하는 방식으로 우회 테스트

```

(base) PS C:\Users\sp9gh\Downloads> $env:KMP_DUPLICATE_LIB_OK="TRUE"
(base) PS C:\Users\sp9gh\Downloads> python .\test_torchaudio.py
테스트 파일 : 101415-3-0-2.wav

=== 원본 정보 ===
waveform shape: torch.Size([1, 192000])
sample_rate: 48000

=== Mono 변환 후 ===
waveform shape: torch.Size([1, 192000])

=== Resampling 후 ===
target sample_rate: 16000
waveform shape: torch.Size([1, 64000])

=== 길이 정보 ===
duration_sec: 4.0

완료 !
(base) PS C:\Users\sp9gh\Downloads>

```

- 테스트 파일 정보
- 파일명: 101415-3-0-2.wav

- 원본 **sample rate**: 48000Hz
- 원본 **waveform shape**: torch.Size([1, 192000])
- 원본 **duration**: 4초
- 전처리 수행
- **mono 변환** (원본파일이 mono였기 때문에 변환 작업이 수행되지 않음)
- **sampling rate**를 16000Hz로 **resampling**
- 전처리 결과
- **resampling 후 waveform shape**: torch.Size([1, 64000])
- **sample 수 감소**: 192000 → 64000
- **duration**은 4초로 유지됨
- 해석
- **sampling rate**를 낮추면 동일한 길이의 오디오라도 **sample** 개수가 감소함
- 입력 형식을 통일하기 위해 **resampling**이 필요할 수 있음
- 대부분 **stereo** 파일이므로 **mono** 변환도 **preprocessing** 단계에서 고려 가능함

### 3. 오디오 데이터 전처리 방향

#### 1) MFCC (Mel-Frequency Cepstral Coefficients)

- 개념
- 사람 귀의 청각 특성을 반영하여 오디오 특징을 압축한 **feature**
- 음성 인식 및 오디오 분류에서 많이 사용됨
- 장점
- 데이터 크기를 줄일 수 있음
- 음성/소리 특징 추출에 효과적
- 계산량이 비교적 적음
- 잡음에 비교적 강한 편
- 단점
- 세부 주파수 정보 손실 가능
- 원본 **waveform** 정보가 많이 압축됨
- 음악·복잡한 환경음에서는 정보 손실 가능성 존재

#### 2) Mel-spectrogram

- 개념
- 시간에 따른 주파수 변화를 이미지 형태로 표현
- 사람 귀의 주파수 인식 방식(**Mel scale**)을 반영
- 장점
- **CNN** 모델에 입력하기 좋음
- 시각화가 직관적
- 환경음 분류에서 많이 사용됨
- 시간/주파수 정보를 비교적 잘 유지
- 단점
- 데이터 크기가 큼
- 계산량 증가 가능
- **MFCC**보다 메모리 사용량 큼

#### 3) Chroma Feature

- 개념
- 음의 높이(**pitch**) 정보를 기반으로 **feature** 추출
- 12개의 **pitch class**(C, C#, D...) 중심으로 표현
- 장점
- 음악 분석 및 화음 분석에 유리
- 음계 정보 표현 가능
- 단점
- 도시 소음(**environmental sound**) 분류에는 적합성이 낮을 수 있음

- pitch 정보가 중요하지 않은 경우 효과 감소

#### 4) Spectral Contrast

- 개념
  - 주파수 대역 간 **peak**와 **valley** 차이를 **feature**로 표현
- 장점
  - 소리의 질감(**texture**) 표현 가능
  - 배경음과 전경음 차이 분석에 도움 가능
- 단점
  - 단독 **feature**로는 성능이 제한적일 수 있음
  - 다른 **feature**와 함께 사용하는 경우가 많음

#### 4. 결과 분석 시 볼 수 있는 지표

##### 1) Accuracy

- 의미: 전체 데이터 중 정답을 맞춘 비율
- 특징
  - 가장 직관적인 지표
  - 클래스 불균형(**class imbalance**) 상황에서는 한계 존재 가능

##### 2) Precision

- 의미: 모델이 양성이라고 예측한 것 중 실제 양성 비율
- 특징: **False Positive**를 줄이는 것이 중요할 때 유용  
예) 잘못된 경보를 줄이고 싶은 경우

##### 3) Recall

- 의미: 실제 양성 중 모델이 찾아낸 비율
- 특징: **False Negative**를 줄이는 것이 중요할 때 유용  
예) 실제 위험 상황을 놓치지 않는 것이 중요한 경우

##### 4) F1-score

- 의미: **Precision**과 **Recall**의 균형을 나타내는 지표
- 특징
  - 데이터 불균형 상황에서 많이 사용
  - **Precision**과 **Recall**을 동시에 고려 가능

##### 5) Confusion Matrix

- 의미
    - 실제 **class**와 예측 **class**를 비교한 표
  - 특징
    - 어떤 **class**끼리 헛갈리는지 확인 가능
    - 모델의 오분류 패턴 분석 가능
- 예) siren을 car\_horn으로 자주 잘못 예측하는지 확인 가능, drilling과 jackhammer를 혼동하는지 분석 가능